National University of Computer & Emerging Sciences, Karachi Computer Science Department

Course Code: SE-3003	Course : Web Engineering Lab

Lab 06: Asp .net Web Forms

What Is ASP.NET?

ASP.NET is a server-side technology used for developing dynamic websites and web applications. It enables developers to create web applications using **HTML**, **CSS**, **and JavaScript**.ASP.NET is the latest version of Active Server Pages, which Microsoft developed to build websites. It is a web application framework released in 2002 and has an extension of .aspx. Some of the major companies that use ASP.NET are :Facebook, Instagram , Twitter , Email and youtube

Life Cycle of ASP.NET: The ASP.NET life cycle is very crucial to developing applications. It includes various stages that help to produce dynamic output.

The Life Cycle of ASP.NET is divided into two categories:

- Application Life Cycle
- Page Life Cycle

Application Life Cycle:

1. Application Start: The web server executes the application start when a user requests an access application. This method sets all global variables by default.

2. Object Creation: Object creation holds all the HTTP Context, HTTP Request, and HTTP Response by the webserver. It also contains information about the request, cookies, and browsing information.

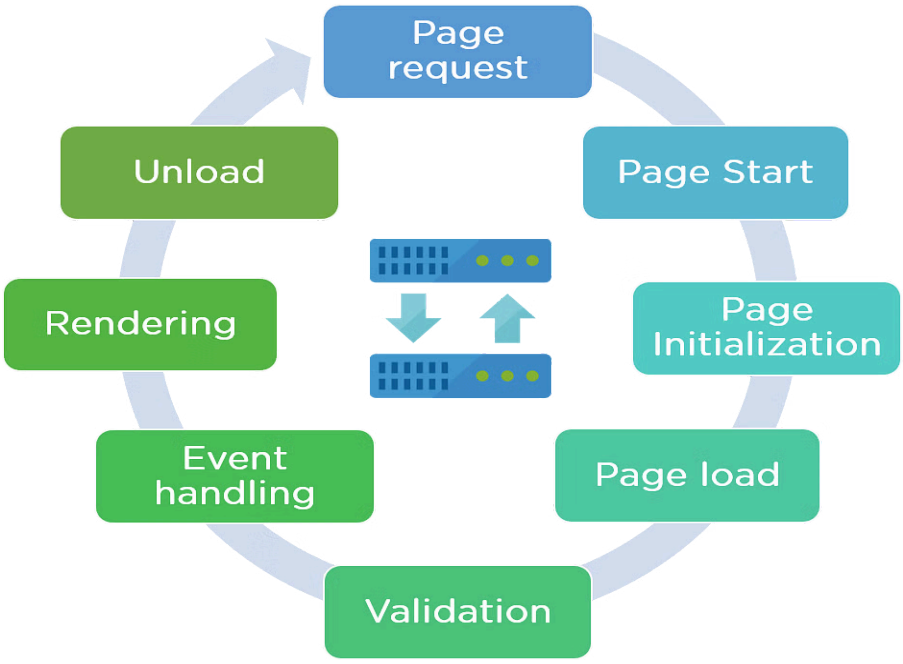
3. HTTP Application: HTTP Application is an object created by the web server. It helps to process all the subsequent information that is sent to the user.

4. Dispose: Dispose is an event that is called before the application is destroyed. It also helps to release manually unmanaged resources when the objects are no longer needed.

5. Application End: Application End is the last stage of the application life cycle. It helps to unload the memory.

Page Life Cycle:

The Page Life Cycle has certain phases that help in writing custom controls and initializing an application.



1. **Page Request:** Page Request is the first step of the page life cycle. When a user request is made, the server checks the request and compiles the pages

2. **Page Start:** Page Start helps in creating two objects: request and response. The request holds all the information that the user sent, while the response contains all the information that is sent back to the user.

3. **Page Initialization:** Page Initialization helps to set all the controls on the pages. It has a separate ID, and it applies themes to the pages in this step.

4. **Page Load:** Page Load helps load all an application's control properties. It also helps to provide information using view state and control state.

5. **Validation:** Validation happens when the execution of an application is successful. It returns two conditions: true and false. If execution is successful, it returns true; otherwise, it returns false.

6. **Event Handling:** Event Handling takes place when the same pages are loaded. It is a response to the validation. When the same page is loaded, a Postback event is called, which checks the page's credentials.

7. **Rendering:** Rendering happens before it sends all the information back to the user. All this information is stored before being sent back.

8. **Unload:** Unload is a process that helps in deleting all unwanted information from the memory once the output is sent to the user.

ASP.NET Web Forms Development Guide

This is a comprehensive guide to developing web applications using ASP.NET Web Forms. It covers the installation of necessary tools, the fundamentals of ASPX pages and controls, and a basic example to get you started.

Installation Guide

1. **Download Visual Studio Community Edition:**
- Go to the official Visual Studio website: https://visualstudio.microsoft.com/vs/community/
- Click the "Download Visual Studio" button for the Community edition.
2. **Run the Installer:**

- Once the download is complete, run the Visual Studio installer.
- Select the ".NET desktop development" workload. This will include the necessary components for Web Forms development.
- You can also choose other workloads if needed, but make sure ".NET desktop development" is selected.
- Click "Install" and wait for the installation to finish.

3. **Launch Visual Studio and Create a Project:**

- Open Visual Studio Community Edition.
- Click "Create a new project."
- In the search bar, type "Web Forms."
- Select "ASP.NET Web Application (.NET Framework)" (make sure it specifically says ".NET Framework").
- Click "Next."
- Give your project a name and location.
- Click "Create."
- In the next dialog, select "Web Forms" as the template.
- Click "Create."

ASPX Pages

- **What they are:** ASPX pages are the core building blocks of Web Forms applications. They are files with the .aspx extension that contain a mix of HTML, server-side code, and ASP.NET controls.
- **How they work:** When a user requests an ASPX page, the server processes the page, executes the server-side code, and generates HTML that is sent back to the user's browser.
- **Structure:**
  - **HTML:** Provides the basic structure and content of the page.
  - **Server-side code:** Written in C# or VB.NET, it handles the logic and data processing.
  - **ASP.NET controls:** Special tags that provide server-side functionality.

How Web Forms Works

- **Event-driven model:** Web Forms applications are based on an event-driven model. Events like button clicks, page loads, and data changes trigger server-side code execution.
- **Page lifecycle:** Each ASPX page goes through a series of stages during its lifecycle, including initialization, loading, event handling, rendering, and unloading.
- **ViewState:** Web Forms uses ViewState to maintain the state of controls between page requests. This allows the page to remember user input and other data.

  Example Code of View State:

.aspx file:

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="ViewStateName.aspx.cs" Inherits="ViewStateName" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>ViewState Name Example</title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
            <asp:Label ID="lblName" runat="server" Text=""></asp:Label>
            <br />
            <asp:TextBox ID="txtName" runat="server"></asp:TextBox>
            <asp:Button ID="btnSaveName" runat="server" Text="Save Name"
OnClick="btnSaveName_Click" />
        </div>
    </form>
</body>
</html>
```

In the .cs file:

```
using System;

namespace ViewStateExample
{
    public partial class ViewStateName : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (ViewState["UserName"] != null)
            {
```

```
            lblName.Text = "Saved Name: " + ViewState["UserName"].ToString();
        }
    }

    protected void btnSaveName_Click(object sender, EventArgs e)
    {
        ViewState["UserName"] = txtName.Text;
        lblName.Text = "Saved Name: " + txtName.Text;
        txtName.Text = ""; // Clear the textbox after saving
    }
  }
}
```

Controls

- **Standard Controls:**
  - **TextBox:** Captures user input.
  - **Label:** Displays text.
  - **Button:** Triggers actions.
  - **DropDownList:** Presents a list of options.
  - **RadioButtonList:** Allows users to select one option from a group.
  - **CheckBoxList:** Allows users to select multiple options from a group.
- **Validation Controls:**
  - **RequiredFieldValidator:** Ensures that a field is not empty.
  - **CompareValidator:** Compares the value of one control to another.
  - **RangeValidator:** Checks if a value falls within a specified range.
  - **RegularExpressionValidator:** Validates a value against a regular expression.
  - **CustomValidator:** Allows you to write custom validation logic.

Example: Creating a Simple Page with Controls

1. **Create a new Web Forms project** (follow the installation steps above).
2. **Open the Default.aspx page.**
3. **Add a TextBox, a Button, and a RequiredFieldValidator control to the page.**
   Code snippet
   ```
   <asp:TextBox ID="txtName" runat="server" />
   <asp:RequiredFieldValidator ID="reqName" runat="server"
   ControlToValidate="txtName" ErrorMessage="Please enter your name."
   />
   <br />
   <asp:Button ID="btnSubmit" runat="server" Text="Submit"
   OnClick="btnSubmit_Click" />
   ```

4. **Double-click the Button control to create an event handler in the code-behind file (Default.aspx.cs).**
5. **In the event handler, write code to display a message based on the user's input.**

   C#
   ```
   protected void btnSubmit_Click(object sender, EventArgs e)
   {
       if (Page.IsValid)
       {
           string name = txtName.Text;
           Response.Write("Hello, " + name + "!");
       }
   }
   ```

6. **Run the application.**

This page will now have a TextBox, a Submit button, and a validator that ensures the TextBox is not empty. When the user clicks the button, the code will check if the page is valid and, if so, display a greeting message.

Event Handling in Web Forms

Event handling is the core mechanism for making Web Forms applications interactive. It allows your code to respond to user actions and other events that occur during the page lifecycle.

- **How it works:**
  - Server controls raise events when specific actions occur (e.g., button clicks, text changes).
  - You can write event handlers in your code-behind file to respond to these events.
  - When an event occurs, the corresponding event handler is executed on the server.
- **Example:**

  Code snippet
  ```
  <asp:Button ID="btnClickMe" runat="server" Text="Click Me"
  OnClick="btnClickMe_Click" />
  ```

  C#
  ```
  protected void btnClickMe_Click(object sender, EventArgs e)
  {
      // Code to execute when the button is clicked
      Response.Write("Button clicked!");
  }
  ```

Postbacks

Postbacks are a fundamental concept in Web Forms. They are the process of sending the entire page back to the server for processing.

- **What it is:**
  - When a user interacts with a server control that causes a postback (e.g., clicking a button), the browser sends the page's current state back to the server.
  - The server processes the page, executes the relevant event handlers, and sends the

updated page back to the browser.

- **Why it's important:**
  - Postbacks enable server-side processing and state management.
  - They allow your code to respond to user actions and update the page dynamically.
- **Example:**
  - The button click example above demonstrates a postback. When the button is clicked, the page is sent back to the server, the btnClickMe_Click event handler is executed, and the updated page is sent back to the browser.

Creating and Implementing Master Pages

Master Pages provide a way to create a consistent layout and design across multiple pages.

- **Creating a Master Page:**
  1. In Visual Studio, right-click your project in the Solution Explorer.
  2. Select "Add" > "New Item."
  3. Choose "Web Form Master Page" and click "Add."
  4. Design the layout of your master page, including headers, footers, and navigation menus.
  5. Use ContentPlaceHolder controls to define regions where content from individual pages will be inserted.
- **Creating a Child Page:**
  1. Right-click your project and select "Add" > "New Item."
  2. Choose "Web Form" and click "Add."
  3. In the "Select a master page" dialog, choose the master page you created.
  4. Add content to the Content controls in your child page, which correspond to the ContentPlaceHolder controls in the master page.
- **Example:**
  - **Master Page (Site.Master):**
    Code snippet

```
<!DOCTYPE html>
<html>
<head runat="server">
    <title></title>
    <asp:ContentPlaceHolder ID="head" runat="server">
    </asp:ContentPlaceHolder>
</head>
<body>
    <form id="form1" runat="server">
        <div id="header">
            <h1>My Website</h1>
        </div>
        <div id="content">
            <asp:ContentPlaceHolder ID="ContentPlaceHolder1"
runat="server">
            </asp:ContentPlaceHolder>
        </div>
        <div id="footer">
            <p>© 2023</p>
        </div>
```

```
    </form>
</body>
</html>
```

- **Child Page (Default.aspx):**

```
Code snippet
<%@ Page Title="" Language="C#" MasterPageFile="~/Site.Master"
AutoEventWireup="true" CodeBehind="Default.aspx.cs"
Inherits="YourProject.Default" %>
<asp:Content ID="Content1" ContentPlaceHolderID="head"
runat="server">
</asp:Content>
<asp:Content ID="Content2"
ContentPlaceHolderID="ContentPlaceHolder1" runat="server">
    <h2>Welcome to my website!</h2>
    <p>This is the content of my default page.</p>
</asp:Content>
```

Creating and Using User Controls

User Controls are reusable UI components that can be embedded in multiple pages.

- **Creating a User Control:**
  1. Right-click your project and select "Add" > "New Item."
  2. Choose "Web User Control" and click "Add."
  3. Design the UI of your user control, including controls and markup.
  4. Add code to the code-behind file to handle events and logic.
- **Using a User Control:**
  1. Register the user control on the page where you want to use it.

```
Code snippet
<%@ Register Src="~/MyUserControl.ascx" TagPrefix="uc"
TagName="MyControl" %>
```

  2. Add an instance of the user control to the page.

```
Code snippet
<uc:MyControl ID="MyControl1" runat="server" />
```

- **Example:**
  - **User Control (MyUserControl.ascx):**

```
Code snippet
<asp:Label ID="lblMessage" runat="server" Text=""></asp:Label>
<asp:Button ID="btnUpdate" runat="server" Text="Update"
OnClick="btnUpdate_Click" />

C#
protected void btnUpdate_Click(object sender, EventArgs e)
{
    lblMessage.Text = "User control updated!";
}
```

- **Page (Default.aspx):**
  Code snippet

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="Default.aspx.cs" Inherits="YourProject.Default" %>
<%@ Register Src="~/MyUserControl.ascx" TagPrefix="uc"
TagName="MyControl" %>
<!DOCTYPE html>
<html>
<head runat="server">
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <uc:MyControl ID="MyControl1" runat="server" />
    </form>
</body>
</html>
```

TASKS:

1. Design a basic sign up page with field validations
2. Create a counter with an increment button using viewstates
3. Design a blog page using web forms with master page containing header and footer for all the other pages.